System Design Notes: Retries, Timeout and Idempotency

Oct 31, 2026 · @Salike Hassan

Why this matters

A reliable system doesn't just hand the user the error that came back from the server. It decides what to do about it. Three concepts work together here: **timeout**, **retry** and **idempotency**.

Running example: a customer places an order, and the orders service calls the payments service to take the payment.

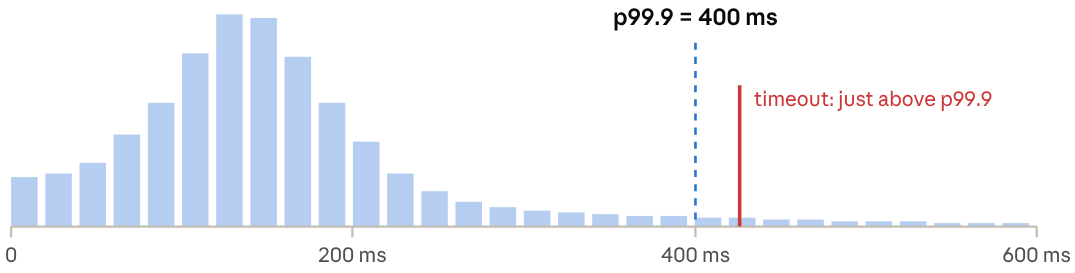
Timeout

Every remote call gets a timeout. The number comes from real latency data, not a guess.

Take the payments call. Look at its latency percentiles, for example **p99.9 = 400 ms**. That tells us how long a good payment can legitimately take.

- **Too short** (100 or 200 ms): healthy payments get cut off, and we trigger retries for nothing.
- **Too long** (1000 ms): we sit waiting on calls that are already lost, and the user and our threads wait with us.
- So the timeout sits just above the p99.9, here a little over 400 ms.

Set the timeout from the latency tail, just above p99.9



Illustrative payments latency. Too short cuts off good payments; too long (1000 ms) waits on calls already lost.

payments latency histogram · illustrative

Most payments finish fast and a thin tail runs out past 400 ms. The tail, not the average, sets the timeout.

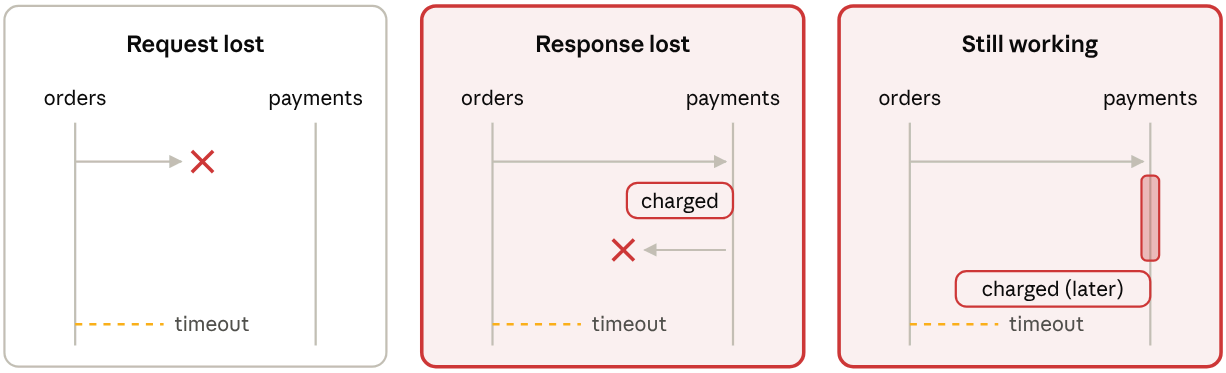
The question to ask: am I ready to wait another 200 ms before I call it a timeout?

What a timeout means

A timeout does not mean the call failed. It means we heard nothing back, and silence can come from three different situations:

1. **Request lost:** payments never got it. No charge.
2. **Response lost:** payments got it and charged the card, but the reply never reached orders.
3. **Still working:** payments is just slow. The charge lands after we gave up.

Same silence, three causes: in 2 of 3 the charge already happened



Red: the card was already charged, so a blind retry double-charges.

timeout cases · 3 call sequences

Orders sees the same timeout in all three, so it cannot tell which one happened.

From the orders side all three look the same (Silen). In **2 of the 3**, the charge already happened, so a blind retry **double-charges**.

That is why a timeout alone is not enough. We need a retry policy, and we need idempotency.

Retry rules

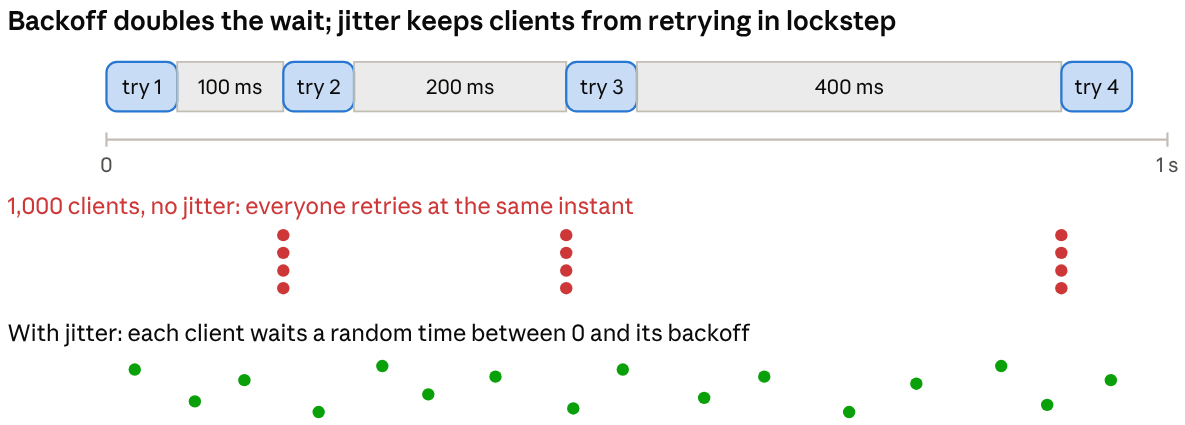
What to retry (transient errors): timeout, connection reset, 503 unavailable.

What not to retry: 400 bad request. The same request will get the same answer every time.

How many times: 2 to 3 attempts at most, because the user is waiting.

How to space them (exponential backoff): the wait doubles each time, for example 100 ms, then 200 ms, then 400 ms. If the server is already struggling, retrying fast only adds more load.

Jitter: each client waits a random time between 0 and the backoff. Without it, 1,000 clients that failed together all retry at the same instants and hit the server in waves. With jitter the retries spread out.



exponential backoff and jitter · 0 to 1 s

The waits (100, 200, 400 ms) fit inside about 1 s. Without jitter the clients retry in three sharp waves. With jitter the same retries spread across the window.

Where to retry

Retrying at every layer multiplies. With 3 attempts at each hop (app, API, orders, payments, ledger, database), one failure becomes 3 x 3 x 3 x 3 x 3 = **243 attempts** at the database.

- **Retry at one layer only:** the first layer, the one that responds to the user. Every layer below it fails fast.
- **Define a retry budget:** retries should stay within about 10% of normal requests. So retries add roughly 10% on top of the traffic, not a multiple of it.
- When the budget is used up, we stop retrying instead of piling more load on a service that is already down.

Retrying at every layer turns one failure into 243 database attempts



every layer	3	9	27	81	243
first layer only	3	3	3	3	3

Attempts each layer receives for one failure, with 3 attempts per retrying layer.

retry amplification · 6 layers

Idempotency keys

A retry is only safe when it doesn't create a new effect. **Idempotent** means doing it twice is the same as doing it once.

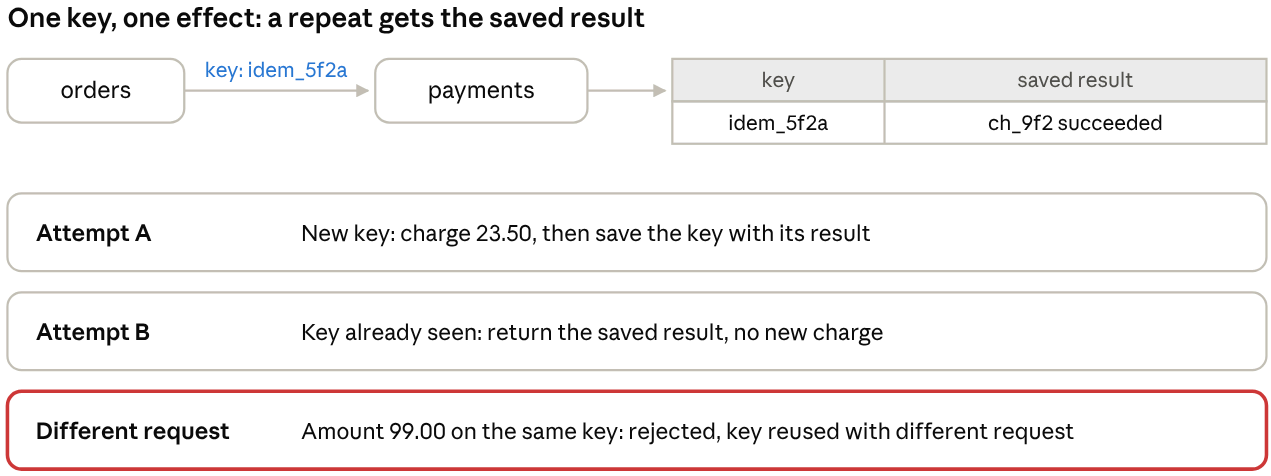
Method	Idempotent
GET	yes
PUT	yes
DELETE	yes
POST /charges	no

Example: a user pays 23.50 for a coffee, something goes wrong, the call is retried, and the card is charged twice. We delivered one coffee but charged two. Rule for any payment: **charge only for what we deliver**. So we never retry a POST blindly.

The fix, as Stripe does it, is an **idempotency key**:

1. Orders creates one key per request (for example `idem_5f2a`) and sends the same key on every attempt of that request.
2. Payments checks the key. If it is new, it does the charge and saves key to result (`ch_9f2 succeeded`). If it has seen the key, it returns the saved result and charges nothing.
3. If two attempts arrive together, the key needs a **lock or a unique constraint**, so only one proceeds.

4. If the same key comes with a different request (for example amount 99.00), it is **rejected**: key reused with different request. It is not treated as a new charge.



idempotency key flow · 3 cases

GET, PUT and DELETE are idempotent by nature. POST is not, so the key is what gives it that property.

Result: delivery 2x, effect 1x.

How Stripe solves it

Stripe takes payments, so it has to be very careful. Its starting point is simple: **networks are unreliable.** Requests get lost, servers fail midway, replies get lost. So the API has to be robust and predictable, and Stripe does that with idempotency.

When	Source	Topic
Feb 2017	Stripe blog	Designing robust and predictable APIs with idempotency
Oct 2017	brandur.org	Implementing Stripe-like Idempotency Keys in Postgres
Today	Stripe API docs	Idempotent requests

What needs to be idempotent, and why

Orders calls payments, and one of three things goes wrong:

1. **Never arrives:** the request is lost.
2. **Fails midway:** the payment server is in limbo.
3. **Reply lost:** the work is done but the answer never comes back.

From the client's side all three are identical: it gets an error and doesn't know what happened. That is why the call must be idempotent: many calls, one effect.

```
f(f(x)) = f(x)
```

- `PUT /domains/blog.example.com`: first call creates it, the second finds it already exists and returns OK. DELETE behaves the same way.
- `POST /v1/charges amount=1000`: call it twice and you get charge #1 and charge #2.

The goal: make it exactly once, from the outside.

Stripe's contract (from the Stripe API docs)

- All POST requests accept an idempotency key. GET and DELETE don't need one.
- The **first result is saved**, status and body, success or failure (even a 500).
- **Same key + different parameters = error.**
- Keys are up to 255 characters. V4 UUIDs are suggested.
- Keys may be pruned after at least 24 hours, so they are not kept forever.
- A validation failure or a concurrent conflict saves nothing, so the client can simply retry.
- The same key also guards concurrency: while one request is running on a key, the key is locked.

Inside the server (Rocket Rides, example app from brandur.org, 2017)

A rider requests a ride. The server does seven steps, in this order:

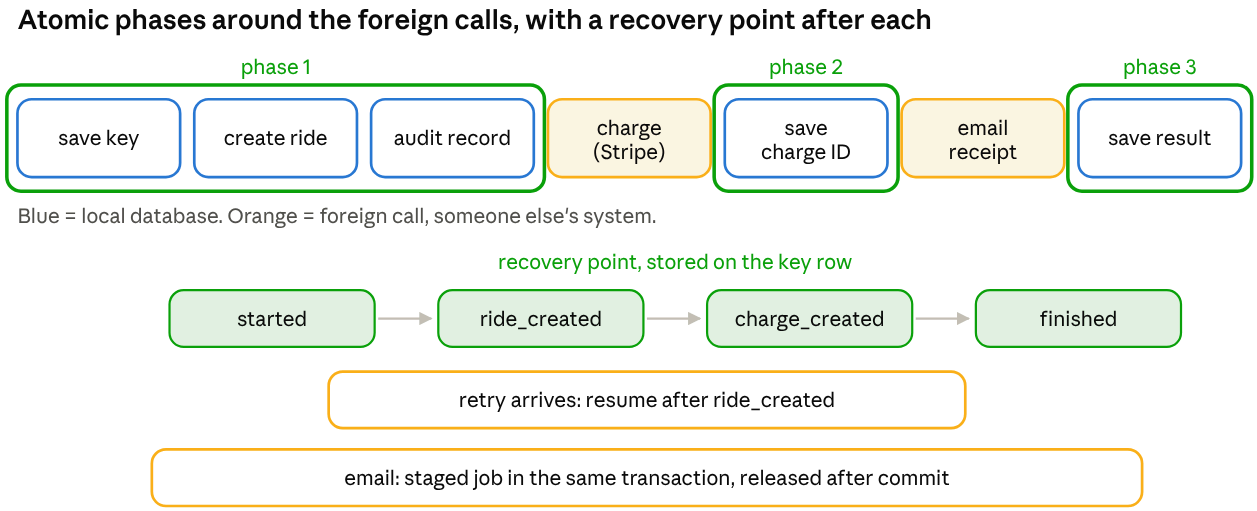
1. Save key (local)
2. Create ride (local)
3. Audit record (local)
4. Charge on Stripe (**foreign**)
5. Save charge ID (local)

6. Email receipt (**foreign**)
7. Save result (local)

Local means our own database: we can wrap it in a transaction. **Foreign** means someone else's system: we can't roll it back. So once we make our first foreign state mutation, we are committed one way or the other.

The answer is **atomic phases** around the foreign calls, with a **recovery point** stored on the key row after each phase. Without a recovery point a retry can't tell where the last attempt stopped.

- **Phase 1:** save key, create ride, audit record. Recovery point: `ride_created`.
- **Phase 2:** save charge ID. Recovery point: `charge_created`.
- **Phase 3:** save result. Recovery point: `finished`.
- **Email:** don't send it inline. Stage it as a job in the same transaction and release it after commit.

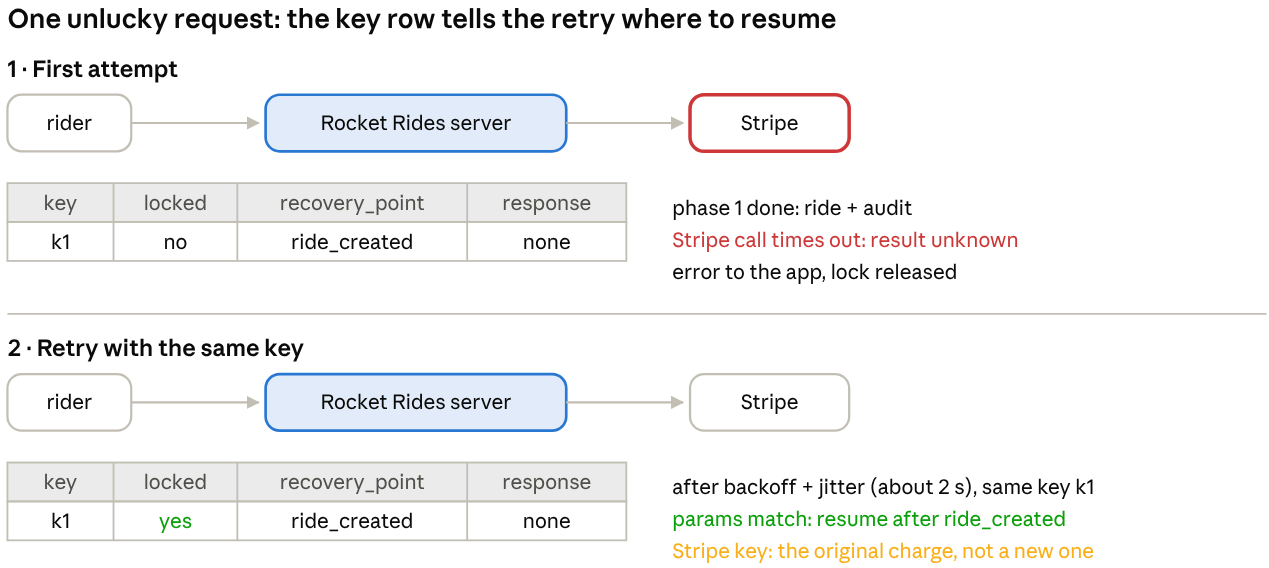


7 steps, 3 atomic phases, recovery points

One unlucky request, end to end

1. The rider's request reaches the server. The key is saved and phase 1 completes. Key row: `k1`, locked no, recovery point `ride_created`, response empty.
2. The server calls Stripe for the charge and gets a timeout. The result is unknown.
3. The charge is ambiguous, so the server returns an error to the app and releases the lock.
4. After a wait (backoff + jitter, about 2 s here), the app retries with the **same key** `k1`.

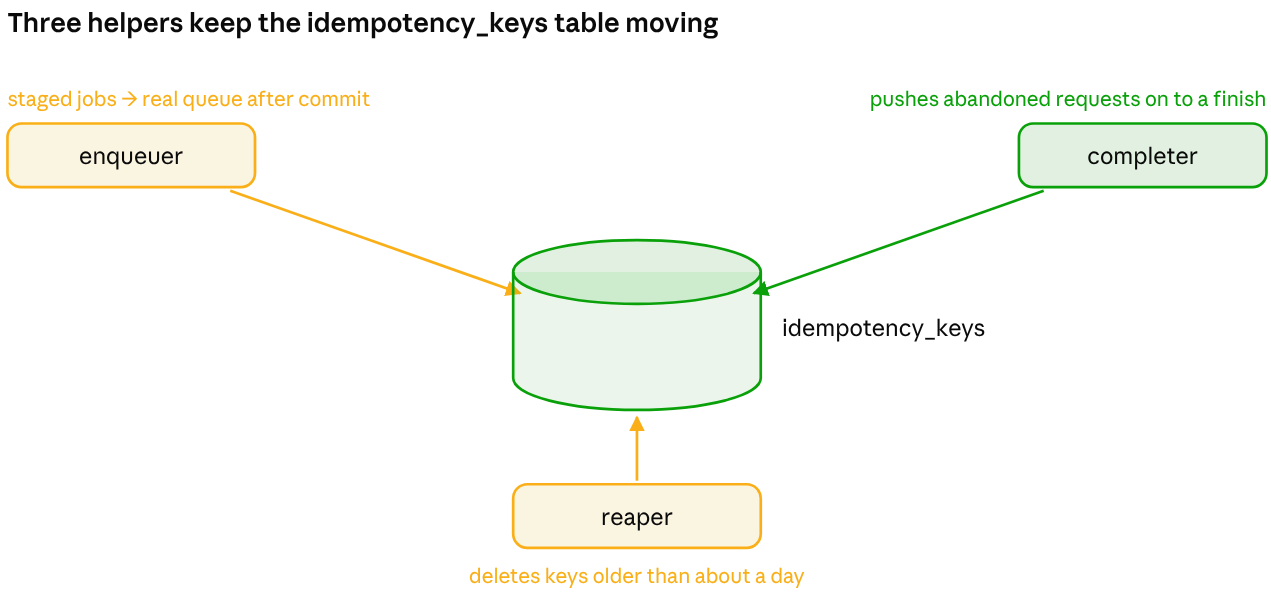
5. The server locks the key, checks that the parameters match, and resumes after `ride_created`. It does not create a second ride.
6. It calls Stripe with Stripe's own idempotency key, so Stripe returns the **original charge**, not a new one. The foreign call is idempotent too.



first attempt and retry · key row state

Three helpers

- **Enqueuer:** moves staged jobs (like the email) to the real queue after commit.
- **Completer:** pushes abandoned requests on to a finish.
- **Reaper:** deletes keys older than about a day.



enqueuer, completer, reaper

Lessons to reuse

- Assume ambiguity. The one safe action is to retry.
- Client-generated keys, and store the full result.
- Compare parameters on retry.
- Atomic phases around foreign calls, with recovery points.
- Backoff + jitter.

In short: Stripe puts an idempotency key on every POST. A retry is not a brand new request. It carries the same key, finds out what happened, and gets the same answer.

Stripe recap

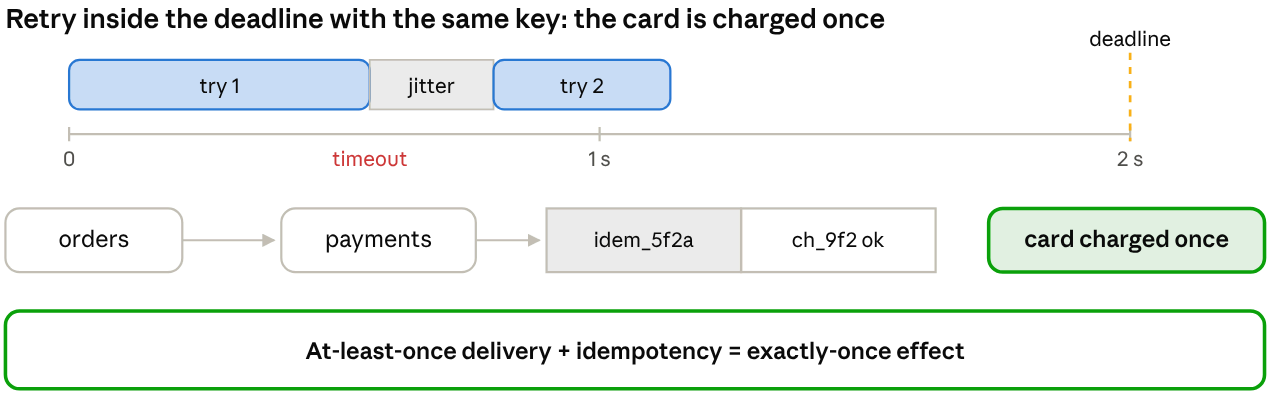
1. Lost request = lost reply, to the client: retry with the same key.
2. First result stored, params compared, keys forgotten after about a day.
3. Atomic phases + recovery points: retries resume, not repeat.

All together

Orders calls payments with a 2 second overall deadline:

1. Try 1 runs until its timeout.
2. Wait a jittered backoff.

3. Try 2 goes out with the **same idempotency key**, still inside the deadline.
4. Payments sees the key, returns the saved result, and the card is charged once.



Bars are not to scale.

timeout + jitter + key inside a 2 s deadline

Timeout, jitter and the retry all fit inside the 2 s deadline, and the key makes the second try safe.

At-least-once delivery + idempotency = exactly-once effect.

Recap

1. Every remote call needs a timeout, taken from real percentiles.
2. A timeout is ambiguous: the work may have already happened.
3. Retry transient errors only, a few times, with jitter, at one layer, inside a budget.
4. Idempotency keys give an exactly-once effect.